



alittlebyte19@gmail.com

Norme di Progetto

Gruppo	aLittleByte (gruppo 19)
Redattori	Eleonora Bellet, Diego Belluz
Verificatori	Leonardo Soligo, Angelica Gastaldello
Destinatari	Prof. Tullio Vardanega, Prof. Riccardo Cardin
Data	-/-/2026

Registro delle Modifiche

Versione	Data	Descrizione	Redattore	Verificatore	Approvatore
0.14.0	20/05/2026	Aggiunta sezione metriche	Diego Belluz	Eleonora Bellet	-
0.13.0	18/05/2026	Aggiunta sezione qualità del software	Diego Belluz	Eleonora Bellet	-
0.12.0	8/05/2026	Struttura dei diari di bordo	Eleonora Bellet	Diego Belluz	-
0.11.2	7/05/2026	Corretti errori di sintassi	Eleonora Bellet	Diego Belluz	-
0.11.1	22/04/2026	Aggiornate e corrette alcune sezioni delle norme	Diego Belluz	Eleonora Bellet	-
0.11.0	19/04/2026	Tracciamento delle azioni, comunicazione, riunioni	Eleonora Bellet	Diego Belluz	-
0.10.0	18/04/2026	Processi Organizzativi (Definizione e scopo, gestione degli sprint, ruoli)	Eleonora Bellet	Diego Belluz	-
0.9.0	17/04/2026	Validazione, Processo di Validazione, Processo di validazione	Eleonora Bellet	Diego Belluz	-
0.8.0	16/04/2026	Controllo della configurazione, Stato della configurazione, Attività di Verifica, Analisi Statica	Eleonora Bellet	Diego Belluz	-
0.7.0	14/04/2026	Denominazione dei documenti, Produzione e Gestione della configurazione	Eleonora Bellet	Diego Belluz	-
0.6.0	11/04/2026	Processi di supporto (scopo, strumenti utilizzati, documenti prodotti, struttura dei documenti)	Eleonora Bellet	Diego Belluz	-
0.5.0	9/04/2026	Processi di sviluppo	Diego Belluz	Eleonora Bellet	-
0.4.0	8/04/2026	Processi di fornitura	Diego Belluz	Eleonora Bellet	-
0.3.0	4/04/2026	Architettura dei processi	Diego Belluz	Eleonora Bellet	-
0.2.0	31/03/2026	Scopo del documento e scopo del prodotto	Eleonora Bellet	Diego Belluz	-
0.1.0	31/03/2026	Struttura del documento	Eleonora Bellet	Diego Belluz	-

Indice

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Scopo del prodotto	4
1.3	Glossario	4
1.4	Riferimenti	5
1.4.1	Normativi	5
1.4.2	Informativi	5
2	Architettura dei processi	5
2.1	Processi Primari	5
2.1.1	Processi di fornitura	5
2.1.1.1	Scopo e obbiettivi	6
2.1.1.2	Attività Operative	6
2.1.1.3	Documentazione Prodotta (Core della Fornitura)	6
2.1.1.4	Comunicazione e Strumenti di Supporto	7
2.1.2	Processi di Sviluppo	7
2.1.2.1	Attività Operative e Ciclo di Vita	7
2.1.2.2	Analisi e Tracciamento dei Requisiti	8
2.1.2.3	Progettazione Architettuale	8
2.1.2.4	Codifica	9
2.2	Processi di supporto	9
2.2.1	Documentazione	9
2.2.1.1	Scopo	9
2.2.1.2	Strumenti Utilizzati	9
2.2.1.3	Documenti Prodotti	10
2.2.1.4	Struttura dei documenti	10
2.2.1.5	Struttura dei verbali	11
2.2.1.6	Struttura dei diari di bordo	11
2.2.1.7	Denominazione dei documenti	12
2.2.1.8	Produzione	12
2.2.2	Gestione della configurazione	13
2.2.2.1	Strumenti utilizzati	13
2.2.2.2	Controllo della configurazione	13
2.2.2.3	Registrazione stato di configurazione	14
2.2.3	Qualifica	14
2.2.3.1	Verifica	14
2.2.3.2	Attività di Verifica	14
2.2.3.3	Analisi Statica	14
2.2.3.4	Analisi Dinamica	15
2.2.3.5	Test Di Unità	15

2.2.3.6	Test di Regressione	15
2.2.3.7	Test di Integrazione	15
2.2.3.8	Test di Sistema	16
2.3	Validazione	16
2.3.0.1	Processo di Validazione	16
2.4	Processi Organizzativi	16
2.4.1	Descrizione e scopo	16
2.4.2	Gestione degli sprint	17
2.4.3	Ruoli	17
2.4.3.1	Responsabile	17
2.4.3.2	Amministratore	18
2.4.3.3	Progettista	18
2.4.3.4	Analista	19
2.4.3.5	Programmatore	19
2.4.3.6	Verificatore	19
2.4.4	Tracciamento delle Ore	20
2.4.5	Tracciamento delle Azioni	20
2.4.6	Comunicazione	20
2.4.6.1	Comunicazioni Interne	20
2.4.6.2	Comunicazioni Esterne	20
2.4.7	Riunioni	21
2.4.7.1	Riunioni Interne	21
2.4.7.2	Riunioni Esterne	21
3	Qualità del software	21
3.1	Qualità del prodotto	22
3.2	Qualità del processo	22
4	Metriche	23
4.1	Metriche per la qualità di processo	23
4.1.1	Processi di Fornitura e Sviluppo	23
4.1.2	Processi di Supporto	24
4.1.3	Processi Organizzativi	25
4.2	Metriche per la qualità di prodotto	25

1 Introduzione

1.1 Scopo del documento

Il presente documento ha l'obiettivo di definire e regolamentare il *Way of Working (WoW)* adottato dal gruppo *ALittleByte* per la realizzazione del progetto didattico. Al suo interno sono specificate le procedure operative, gli strumenti adottati e le regole di collaborazione interne al team, al fine di garantire un approccio strutturato, efficiente e tracciabile per l'intero ciclo di vita del progetto Nexum.

La stesura del seguente documento segue un approccio incrementale: le norme qui descritte verranno costantemente aggiornate e affinate nel tempo, supportando il miglioramento continuo dei processi organizzativi, di sviluppo e di supporto del gruppo.

1.2 Scopo del prodotto

Il prodotto da realizzare consiste in un'estensione innovativa per "*Nexum*", una piattaforma digitale sviluppata dall'azienda proponente *Eggon*, dedicata alla comunicazione interna e alla gestione HR. Nello specifico, il progetto mira a potenziare l'ecosistema Nexum attraverso lo sviluppo e l'integrazione di due moduli basati sull'Intelligenza Artificiale:

- **AI Assistant Generativo:** Un modulo dedicato agli utenti amministrativi (dashboard) che permette la creazione autonoma di contenuti aziendali (titolo, testo, immagini di copertina) a partire da un prompt, garantendo l'adeguamento automatico del tono e dello stile del messaggio.
- **AI Co-Pilot per Consulenti del Lavoro (CdL):** Un sistema avanzato in grado di riconoscere automaticamente la tipologia di documenti caricati (es. cedolini, comunicazioni), estrarne i destinatari tramite tecniche di OCR ed **Entity Resolution**, e automatizzare lo smistamento e il dispaccio massivo.

L'obiettivo finale è migliorare l'efficienza dei processi HR e semplificare l'interoperabilità tra le aziende e gli studi dei Consulenti del Lavoro. Per fare ciò il gruppo *ALittleByte* si impegna a completare il progetto entro la data fissata 10 Settembre 2026, con un costo totale di budget di 12.575 €.

Per ulteriori informazioni si rimanda al documento Analisi dei Requisiti.

1.3 Glossario

Al fine di prevenire qualsiasi ambiguità linguistica o concettuale tra i membri del team e gli stakeholder, è stato redatto un documento separato denominato Glossario. Esso contiene la spiegazione puntuale di tutti i termini tecnici, di dominio (es. LLM, OCR, Dispaccio, ecc.) e degli acronimi utilizzati all'interno della documentazione di progetto.

1.4 Riferimenti

1.4.1 Normativi

- **Regolamento del Progetto Didattico:** Anno accademico 2025/2026, Corso di Ingegneria del Software, Università di Padova
 - <https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
- **Capitolato d'appalto C5:** Nexum - Eggon S.r.l.
 - <https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf>

1.4.2 Informativi

- Glossario:

2 Architettura dei processi

Per la realizzazione di un prodotto di qualità che soddisfi le esigenze del cliente e degli utenti, il gruppo aLittleByte ha deciso di adottare un'architettura specifica e suddividere i vari processi di costruzione del software in tre macro-aree:

- Processi primari
- Processi di supporto
- Processi organizzativi

2.1 Processi Primari

In questa sezione viene descritto come nasce il software. I processi primari indicano tutte quelle attività che sono direttamente coinvolte nella creazione del prodotto software, è possibile classificare i processi primari in:

- Processi di fornitura
- Processi di sviluppo

2.1.1 Processi di fornitura

I processi di fornitura comprendono tutte le attività preliminari che permettono di avviare lo sviluppo del progetto, in particolar modo la pianificazione: suddivisione del lavoro in fasi e milestone come rtb e pb, l'analisi: come vengono analizzati i capitolati, la stima: come si stimano vari costi e stime.

2.1.1.1 Scopo e obbiettivi Il processo di fornitura definisce le procedure e le attività che il gruppo mette in atto per stabilire un accordo operativo con la proponente (*Eggon*). L'obiettivo è trasformare le necessità espresse nel capitolato C5 in un insieme di requisiti tecnici e vincoli contrattuali ben definiti. In questa fase si stabiliscono le basi metodologiche del "*Way of Working*" del team, garantendo che ogni attività sia pianificata, tracciabile e verificabile.

2.1.1.2 Attività Operative Il team si impegna nelle seguenti attività chiave:

- **Analisi e Valutazione del Capitolato:** Studio critico della proposta C5 per identificare rischi tecnologici, opportunità di apprendimento e coerenza con le competenze del team.
- **Stima e Pianificazione:** Definizione del budget (12.575€) e delle scadenze temporali (consegna finale entro il 10 settembre 2026), con la suddivisione del lavoro in cicli iterativi (Sprint).
- **Negoziazione e Definizione dell'MVP:** Collaborazione con *Eggon* per identificare il *Minimum Viable Product*, garantendo che le funzionalità core (AI Assistant e Co-Pilot) siano priorizzate correttamente.
- **Accertamento della Qualità:** Definizione degli obiettivi di qualità per i processi e per il prodotto finale.

2.1.1.3 Documentazione Prodotta (Core della Fornitura) La documentazione non è solo un onere burocratico, ma lo strumento principale per garantire la trasparenza verso i committenti e la coerenza interna. Di seguito sono elencati i documenti redatti e mantenuti dal team:

- **Analisi dei Requisiti:** Documento cardine che formalizza le funzionalità del sistema. Include la modellazione degli attori (Redattore, Analista dati, Consulente del lavoro), la descrizione dei casi d'uso (UC) e la classificazione dei requisiti funzionali, qualitativi e di vincolo.
- **Norme di Progetto:** Definiscono il regolamento interno del gruppo. Contengono gli standard di codifica, le procedure di gestione dei ticket su GitHub, le convenzioni di naming e le metodologie di lavoro adottate.
- **Piano di Progetto:** Strumento di gestione manageriale che descrive l'allocazione delle risorse, l'analisi dei rischi e il preventivo economico. Viene aggiornato periodicamente con il consuntivo delle ore per monitorare lo stato del budget.
- **Piano di Qualifica:** Specifica la strategia per garantire che il software sia privo di difetti critici. Definisce le metriche (es. *Code Coverage* $\geq 80\%$) e i test (unità, integrazione, sistema) necessari alla validazione.
- **Glossario:** Raccolta di termini tecnici e acronimi per eliminare ambiguità terminologiche tra sviluppatori, committenti e utenti finali.

- **Specifica Tecnica:** Documento di design che descrive l'architettura di sistema (modello C4), i diagrammi delle classi UML e i design pattern scelti per l'implementazione.
- **Manuale Utente:** Guida operativa destinata agli utenti finali per facilitare l'onboarding sulla piattaforma Nexum e spiegare l'utilizzo degli assistenti AI.
- **Lettera di Candidatura e Presentazione:** Documenti formali per l'ufficializzazione della partecipazione all'appalto e per la consegna dei vari rilasci (RTB e PB).
- **Verbali (Interni ed Esterni):** Report che tracciano l'ordine del giorno, le discussioni e le decisioni prese durante le riunioni di team o con i referenti di Eggon.

2.1.1.4 Comunicazione e Strumenti di Supporto Per la gestione dei processi di fornitura, il gruppo si avvale di:

- **GitHub e GitLab:** Per il versionamento della documentazione tramite *Pull Request* e per l'integrazione con il codice preesistente.
- **Comunicazione Ufficiale:** L'interfaccia con Eggon avviene tramite incontri bisettimanali e via email/Telegram per comunicazioni rapide.
- **Tracciamento:** Ogni decisione rilevante viene convertita in una *issue* su GitHub per garantirne l'assegnazione e il completamento.

2.1.2 Processi di Sviluppo

Questa sezione codifica l'insieme delle pratiche ingegneristiche e operative indispensabili per costruire materialmente la piattaforma software stand-alone richiesta dal capitolato. L'obiettivo è tradurre i requisiti concordati in un applicativo funzionante e architetturealmente indipendente dal sistema originario (*Nexum*), passando attraverso fasi rigorose di design, programmazione e collaudo.

2.1.2.1 Attività Operative e Ciclo di Vita Il ciclo di sviluppo per l'integrazione dei modelli di Intelligenza Artificiale richiede una sequenza logica di operazioni ingegneristiche specifiche:

- **Analisi e Modellazione del Dominio:** Studio sistemistico per separare nettamente i flussi di lavoro aziendali (creazione contenuti) da quelli consulenziali (gestione documentale massiva).
- **Progettazione Architettureale:** Definizione di un'infrastruttura *stand-alone* che comunichi in modo asincrono con i moduli esterni di Intelligenza Artificiale (AI Post Generator e AI Analyst), per non bloccare l'interfaccia utente durante le operazioni *time-consuming*.
- **Sviluppo Modulare:** L'applicativo viene frammentato in macro-componenti indipendenti (es. modulo di rating/feedback, modulo di split documentale, dashboard analitica).

- **Validazione "Human-in-the-loop":** Implementazione di flussi di sviluppo che prevedano sempre un passaggio di validazione umana per le operazioni AI a bassa confidenza (es. revisione dei destinatari estratti o modifica del testo generato).
- **Collaudo e Rilascio:** Esecuzione di test formali sulle metriche di qualità (es. tempi di risposta, tasso di successo dell'AI) prima dell'approvazione finale.

2.1.2.2 Analisi e Tracciamento dei Requisiti Fase nevralgica per la definizione del perimetro del prodotto. Per garantire chiarezza e tracciabilità, il team adotta standard di modellazione e nomenclatura precisi per attori e casi d'uso.

- **Modellazione degli Attori:** Il sistema riconosce due categorie di attori per separare le responsabilità di business da quelle computazionali:
 - *Utenti Operativi (Primari):* Utenti umani che innescano le operazioni (Amministratore, Redattore, Analista dati, Consulente del lavoro).
 - *Moduli Esterni (Secondari):* Entità di sistema chiamate in causa per completare l'operazione (AI Post Generator, AI Analyst).
- **Nomenclatura dei Casi d'Uso (Use Case):** Il team adotta una convenzione basata sui prefissi per separare logicamente le aree dell'applicativo:
 - UC-[Numero] identifica i casi d'uso relativi all'**AI Assistant Generativo** (es. *UC-1*).
 - UC-CO-[Numero] identifica i casi d'uso relativi all'**AI Co-Pilot** (es. *UC-CO-1*).
 - [Caso Padre].[Sottocaso] indica le scomposizioni logiche (es. *UC-1.1* per un sottocaso di *UC-1*).
- **Struttura Descrittiva:** Ogni caso d'uso documentato deve presentare una scheda strutturata contenente: Attori coinvolti, Pre e Post-condizioni, Scenario Principale (sequenza ottimale numerata), Scenari Alternativi o di Errore (identificati con codici specifici come UC-1.E1), e Relazioni (Inclusioni, Estensioni).

2.1.2.3 Progettazione Architeturale La progettazione traduce le specifiche in un'infrastruttura flessibile e scalabile.

- **Disaccoppiamento e Asincronia:** Data la natura *data-driven* e i tempi di latenza dell'AI, l'architettura deve garantire l'esecuzione asincrona delle operazioni pesanti (es. elaborazione OCR o entity resolution).
- **Modellazione Visiva:** La struttura del codice viene rappresentata tramite diagrammi UML per la definizione dei casi d'uso e la struttura statica. Si adotta inoltre un approccio a componenti per isolare le funzionalità trasversali (raccolta log, gestione ruoli, telemetria).
- **Sicurezza e Governance:** L'architettura deve supportare by-design workflow tracciabili (audit trail), separazione dei ruoli e policy di conservazione, fondamentali per trattare documenti sensibili come i cedolini.

2.1.2.4 Codifica Questa è la fase di traduzione dei modelli architetturali in sorgente eseguibile.

- **Modularità e Riuso:** Suddivisione del codice in moduli con responsabilità singola (es. libreria dei prompt separata dal motore di generazione). Applicazione di pattern architetturali per evitare dipendenze dirette e favorire la manutenibilità.
- **Gestione degli Errori e Resilienza:** Implementazione di meccanismi robusti per la gestione di anomalie, retry di rete e fallback nel caso in cui i servizi AI esterni non siano temporaneamente disponibili (come mappato negli scenari alternativi dei casi d'uso).
- **Tracciabilità e Qualità:** Versionamento continuo del codice, revisione tra pari (*Code Review*) e test automatizzati per validare non solo le logiche applicative, ma anche l'integrazione corretta dei parametri (tono, stile, metadati) passati ai core di intelligenza artificiale.

2.2 Processi di supporto

I processi di supporto comprendono tutte le attività, i metodi e gli strumenti che non sono direttamente legati alla scrittura del codice, ma che risultano indispensabili per sostenere i processi primari di sviluppo e fornitura. Per il nostro team, definire questi processi significa stabilire delle regole di base condivise, necessarie per collaborare in modo efficiente ed evitare disorganizzazione durante le varie fasi del progetto didattico. Ciò ci permette, a livello pratico, di garantire la tracciabilità di ogni decisione tecnica e organizzativa presa dal gruppo e di assicurare che tutto il materiale prodotto (documenti, codice, issue) sia costantemente coerente, aggiornato e facilmente accessibile a tutti i membri del team e al committente.

2.2.1 Documentazione

2.2.1.1 Scopo Lo scopo del processo di documentazione è stabilire regole e metodologie pratiche per la creazione, la gestione e l'aggiornamento continuo dei documenti di progetto. Per il nostro gruppo, impegnarci a produrre materiale che si mantenga costantemente chiaro, accessibile e aggiornato ci permette di agevolare e rendere trasparente la comunicazione interna tra i membri del gruppo. Inoltre, una repository ben strutturata facilita notevolmente il confronto e l'allineamento con gli stakeholder esterni e l'azienda *Eggon*. Questo approccio rigoroso garantisce un'accurata tracciabilità delle decisioni organizzative e progettuali prese nel corso dello sviluppo, assicurando al contempo che l'intero progetto venga gestito in piena conformità agli standard di qualità che ci siamo prefissati di rispettare.

2.2.1.2 Strumenti Utilizzati Per gestire in modo efficiente la stesura, la revisione e il versionamento della documentazione, il gruppo *aLittleByte* ha scelto di utilizzare i seguenti strumenti:

- **GitHub:** Rappresenta la nostra piattaforma principale per l'archiviazione e il tracciamento storico dei file. Sfruttiamo le funzionalità di branching e pull request per

garantire che ogni modifica venga revisionata in modo asincrono da un altro membro prima di essere integrata nella versione definitiva. Utilizziamo inoltre le issue per pianificare e monitorare le attività legate alla documentazione.

- **Latex:** Scelto per la stesura di tutti i documenti tecnici e formali. Questo strumento ci garantisce un’impaginazione professionale, rigorosa e visivamente uniforme in tutto il progetto, grazie all’impiego di template e stili predefiniti condivisi all’interno del gruppo.

2.2.1.3 Documenti Prodotti I documenti ufficiali previsti per il progetto, i quali saranno costantemente aggiornati e mantenuti sotto controllo di versione all’interno del repository, sono i seguenti:

- **Analisi dei requisiti:** Il documento in cui vengono estrapolati, classificati e descritti tutti i requisiti del sistema (funzionali e non), i casi d’uso individuati e i rispettivi criteri per l’accettazione finale.
- **Norme di progetto:** Il documento in esame, avente lo scopo di codificare il nostro Way of Working, illustrando nel dettaglio le metodologie operative, gli strumenti scelti e le regole di collaborazione interne al team.
- **Piano di progetto:** Il piano strategico che scandisce le tempistiche di sviluppo, regola l’allocazione delle risorse umane ed economiche e pianifica le varie fasi di avanzamento del lavoro.
- **Piano di qualifica:** Il documento volto a fissare le direttive per l’assicurazione della qualità. Al suo interno vengono delineate le metriche di valutazione e le procedure di testing, con l’obiettivo specifico di garantire una copertura del codice non inferiore all’80%.
- **Glossario:** Uno strumento di supporto testuale che definisce in modo inequivocabile la terminologia specifica, tecnica e di dominio, oltre agli acronimi impiegati in tutta la documentazione.
- **Lettera di Presentazione:** La documentazione formale di accompagnamento, debitamente firmata, necessaria per certificare e presentare ufficialmente il RTB al committente.
- **Verbali:** I Verbali (interni ed esterni) tengono traccia degli incontri e delle decisioni sia tra i membri del gruppo e che tra il gruppo e la proponente.

2.2.1.4 Struttura dei documenti Tutti i documenti (eccetto i verbali e i diari di bordo) hanno la seguente struttura generale:

- **Copertina:** Riporta il logo, l’indirizzo email del gruppo, il titolo del documento, il nome del gruppo e i metadati essenziali (redattori, verificatori, destinatari e data).
- **Registro delle modifiche:** Tabella posta all’inizio del file per il tracciamento delle versioni. Per ogni aggiornamento indica: numero di versione, data, descrizione della modifica, redattore e approvatore/verificatore.

- **Indice:** Elenco gerarchico dei contenuti con rinvii numerici alle pagine, per favorire una navigazione rapida e mirata.
- **Introduzione:** Contiene lo scopo del documento, glossario e riferimenti.
- **Contenuto:** Corpo centrale del testo organizzato in sezioni e sottosezioni numerate (es. 1, 1.1), redatto in stile chiaro e esecutivo.

2.2.1.5 Struttura dei verbali

- **Copertina:** Riporta gli elementi visivi e identificativi del gruppo, il titolo (specificando se verbale interno o esterno e la data), i ruoli assunti per la stesura (redattore e verificatore) e l'elenco dei destinatari.
- **Indice:** Elenco cliccabile delle sezioni per facilitare la navigazione del documento.
- **Informazioni generali:** Tabella riassuntiva contenente i dettagli logistici dell'incontro: luogo o piattaforma utilizzata (Discord), orario di inizio e fine, e l'elenco puntuale dei partecipanti.
- **Ordine del giorno:** Elenco sintetico degli argomenti pianificati e discussi durante la riunione.
- **Attività svolte:** Resoconto testuale, suddiviso in sottoparagrafi, che descrive in modo oggettivo le discussioni affrontate, le problematiche analizzate e le decisioni prese per ciascun punto all'ordine del giorno.
- **To-Do List:** Sezione conclusiva dedicata al tracciamento delle azioni da intraprendere (task) a seguito della riunione, utili per assegnare le responsabilità future, collegate alle rispettive issues.

2.2.1.6 Struttura dei diari di bordo I Diari di Bordo sono delle diapositive realizzate per gli incontri settimanali con i docenti per vedere lo stato di avanzamento del lavoro. Il loro scopo principale è condividere rapidamente con i docenti e i colleghi i progressi effettuati e gli ostacoli incontrati dal gruppo *aLittleByte*.

Sono documenti brevi che trattano dei seguenti argomenti:

- **Copertina:** Riporta le informazioni identificative di base, ovvero il logo del team, la data della presentatione, il numero del gruppo e i nomi dei componenti.
- **Cosa abbiamo fatto:** Un riepilogo sintetico e puntato delle attività concluse nell'ultimo periodo.
- **Cosa faremo:** La pianificazione operativa per i giorni successivi, indicando i prossimi *task* da completare.
- **Difficoltà e dubbi:** Una sezione in cui vengono elencati i problemi affrontati dal team e i dubbi da sottoporre all'attenzione del corpo docente.

Questa suddivisione in sezioni permette di ottimizzare i tempi della presentazione, garantendo che i messaggi chiave arrivino in modo chiaro e che ci si possa concentrare immediatamente sulla discussione e risoluzione dei problemi.

2.2.1.7 Denominazione dei documenti Il gruppo *aLittleByte* ha adottato una rigorosa politica di denominazione univoca per tutta la documentazione. Tale scelta metodologica è volta a preservare l'uniformità del repository e a ottimizzare le procedure di ricerca e consultazione dei file.

- **Verbali:** I verbali hanno la seguente denominazione:

`<Tipo>.<Data>`

Dove `<Tipo>` rappresenta:

- **Verbale_Interno**
- **Verbale_Esterno**

Mentre `<Data>` è la data della riunione scritta nel formato `AAAA/MM/GG`. Ad esempio, `Verbale_Interno_2026_04_13.pdf` rappresenta il verbale interno del giorno 13/04/2026.

- **Diari di Bordo:** I diari di bordo hanno la seguente denominazione: da completare
- **Documenti:** I restanti documenti ufficiali seguono una convenzione di denominazione esplicita basata sul loro contenuto, come ad esempio *Norme di Progetto.pdf* o *Analisi dei Requisiti.pdf*.

2.2.1.8 Produzione Il ciclo di vita di ogni documento prodotto dal team è regolato da una procedura strutturata, volta a massimizzare la qualità, l'uniformità e la tracciabilità delle informazioni. Il processo si articola nelle seguenti fasi:

- **Pianificazione:** In corrispondenza delle scadenze di progetto, il gruppo stabilisce quali documenti produrre o aggiornare. Per ogni compito vengono designati un redattore e un verificatore, scelti nel rispetto dei ruoli previsti dal *Way of Working* e garantendo l'assenza di conflitti d'interesse.
- **Creazione del branch:** Prima di iniziare la stesura, viene aperto un *branch* dedicato nel repository GitHub. Il gruppo *aLittleByte* ha stabilito di creare un ramo separato per ogni singolo file, inclusi i verbali e i diari di bordo, al fine di garantire una gestione granulare delle modifiche e un isolamento totale del lavoro rispetto al ramo principale.
- **Redazione:** Il redattore incaricato sviluppa il contenuto seguendo i template e le direttive stilistiche definite nelle presenti Norme di Progetto. La scrittura avviene tramite gli strumenti appropriati (ad esempio *LaTeX*), assicurando un output professionale e coerente.
- **Revisione:** Terminata la stesura, il documento viene sottoposto a una verifica asincrona. Il verificatore analizza il file tramite una *pull request* su GitHub, segnalando eventuali correzioni necessarie o suggerimenti direttamente nei commenti della piattaforma.

- **Integrazione e Approvazione:** Una volta superata la revisione e ottenuta l'approvazione formale dal Responsabile, il contenuto viene integrato nel *branch main* del repository. In questa fase viene consolidata la versione finale nel registro delle modifiche.

2.2.2 Gestione della configurazione

Il processo di Gestione della Configurazione ha l'obiettivo di tracciare, organizzare e controllare in modo sistematico l'evoluzione di tutti gli artefatti di progetto (codice sorgente, documentazione, script). Adottare questo processo permette al gruppo *aLittleByte* di prevenire conflitti, recuperare versioni precedenti in caso di errore e garantire che ogni rilascio sia coerente e verificabile.

2.2.2.1 Strumenti utilizzati Al fine di implementare correttamente tale processo, il gruppo ha selezionato i seguenti strumenti tecnici:

- **Git:** Software per il controllo di versione distribuito che permette di monitorare ogni variazione apportata ai file sorgente e alla documentazione.
- **GitHub:** Servizio di hosting basato su cloud utilizzato per la gestione centralizzata del repository. Viene impiegato non solo per l'archiviazione, ma anche per la pianificazione operativa tramite le *Issue* e per la revisione del lavoro attraverso le *Pull Request*.

2.2.2.2 Controllo della configurazione Il controllo della configurazione è l'attività preposta alla gestione, tracciabilità e autorizzazione di qualsiasi proposta di modifica del progetto. Per tracciare in modo controllato tutte le modifiche, che vengono approvate o meno, il gruppo *aLittleByte* sfrutta le funzionalità native di GitHub, strutturando il processo attraverso l'uso rigoroso di *Issue* e *Pull Request*:

- **Issue:** Ogni singola attività (che si tratti della stesura di un documento, dello sviluppo di una funzionalità, della correzione di un errore o anche semplicemente contattare l'azienda) deve essere preventivamente tracciata tramite l'apertura di un'*Issue*. Quest'ultima viene assegnata a uno o più membri incaricati di portarla a termine. Per facilitarne la classificazione, ogni *Issue* è identificata da una nomenclatura univoca (ad esempio per la stesura dei documenti viene usato il tag *documentazione*).
- **Pull Request:** Rappresenta il meccanismo esclusivo per l'integrazione di qualsiasi modifica all'interno del repository. Al fine di mantenere un elevato standard qualitativo e prevenire errori, il team ha stabilito che nessun componente è autorizzato a effettuare modifiche dirette sul ramo principale (*main*). Perciò ogni integrazione deve necessariamente passare per una *Pull Request*, la quale richiede obbligatoriamente un processo di revisione incrociata e può essere inserita nel *main* solo a seguito di almeno una revisione positiva da parte di altri membri del gruppo.

2.2.2.3 Registrazione stato di configurazione Ogni documento presenta un *Registro delle modifiche* (eccetto i verbali) per tenere traccia di tutte le modifiche svolte. Il sistema di versionamento è il seguente:

X.Y.Z

Dove:

- **X:** Subisce un incremento solo quando il file viene approvato dal membro del gruppo incaricato del ruolo di approvatore;
- **Y:** Indica un avanzamento significativo, come l'aggiunta di interi capitoli o moduli software che hanno superato positivamente la fase di verifica.
- **Z:** Rappresenta modifiche di dettaglio, correzioni di refusi o aggiornamenti minimi eseguiti durante la stesura o lo sviluppo.

2.2.3 Qualifica

2.2.3.1 Verifica Le attività di verifica hanno lo scopo di garantire l'assenza di difetti e la totale aderenza di ogni componente prodotta (sia essa documentale o codice sorgente) rispetto alle specifiche e ai vincoli stabiliti. Applicando i principi dell'Ingegneria del Software, questa pratica metodologica mira a soddisfare la seguente domanda: *"Did I build the system right?"*.

L'intera reportistica riguardante i controlli effettuati, i parametri misurati e gli esiti dei collaudi è raccolta e analizzata all'interno del **Piano di Qualifica**.

2.2.3.2 Attività di Verifica La verifica interviene nelle singole fasi del nostro progetto, con lo scopo di assicurarci che il lavoro appena svolto non abbia introdotto errori. In termini pratici, questa attività serve a controllare che il risultato ottenuto alla fine di una determinata fase sia sempre coerente con le informazioni e i requisiti che avevamo in partenza.

In questa prima parte del lavoro, il gruppo *aLittleByte* si è dedicato principalmente alla verifica della documentazione. Tramite revisioni incrociate tra i membri, ci siamo assicurati che i contenuti fossero corretti e che i testi rispettassero le regole grammaticali, sintattiche e di formattazione che ci siamo dati nelle presenti Norme di Progetto.

Per quanto riguarda invece la verifica del codice sorgente e del prodotto software, ce ne occuperemo nel dettaglio una volta superata la revisione *RTB*. In ogni caso, tutte le decisioni prese, i metodi di controllo utilizzati e i risultati di queste verifiche verranno sempre riportati in modo esaustivo nel documento **Piano di Qualifica**.

2.2.3.3 Analisi Statica L'analisi statica è una tecnica di verifica che viene applicata all'oggetto di studio senza richiederne l'esecuzione. Questo approccio ci permette di individuare anomalie e difetti fin dalle primissime fasi del ciclo di vita del software, ben prima che la scrittura del codice sia ultimata.

Questo tipo di analisi può avvenire tramite metodi formali o, più comunemente nel nostro caso, attraverso tecniche di lettura. Il nostro team adotta principalmente due metodologie:

- **Walkthrough:** Consiste in una revisione estensiva e senza preconcetti. Il verificatore analizza il prodotto passo dopo passo, simulandone mentalmente la logica e verificando che le regole prestabilite siano rispettate. È un approccio molto approfondito ma dispendioso in termini di tempo, e richiede un intervento interamente manuale.
- **Inspection:** Si tratta di un controllo mirato e guidato dall'esperienza. Il verificatore sa già quali sono le criticità più comuni e utilizza delle liste di controllo (*checklist*) specifiche per scovarle. Pur essendo meno estesa del *walkthrough*, l'ispezione è più rapida e permette di standardizzare la ricerca dei difetti.

Al termine di ogni controllo, le attività svolte e i problemi riscontrati vengono opportunamente documentati. Gli errori vengono discussi in modo costruttivo tra il verificatore e l'autore del documento (o del codice); tuttavia, l'applicazione materiale delle correzioni rimane sempre di competenza dell'autore originale, al fine di garantire coerenza e responsabilità. Tutto ciò viene documentato all'interno della *Pull Request*, che in caso di errori non viene approvata, e nel relativo documento.

2.2.3.4 Analisi Dinamica A differenza della tecnica statica, l'analisi dinamica presuppone l'esecuzione dell'oggetto sottoposto a verifica. Tale approccio esamina le reazioni di specifiche porzioni del software applicandovi un numero definito di "casi prova". Questi test devono possedere i requisiti di ripetibilità e automazione.

Ogni sessione di test è definita dai seguenti stati:

- **NI:** non implementato
- **I:** implementato
- **S:** superato
- **NS:** non superato

2.2.3.5 Test Di Unità I test d'unità si focalizzano sulla verifica isolata di singole componenti software, come funzioni, classi o metodi, separandole dal resto dell'architettura di sistema.

Il fine di questa attività risiede nel confermare che ogni unità generi i risultati previsti partendo da input determinati. Questo processo consente di intercettare precocemente difetti di implementazione o errori nella logica funzionale durante le fasi iniziali della progettazione.

2.2.3.6 Test di Regression L'esecuzione dei test di regressione è finalizzata ad accertare che interventi correttivi o evolutivi sul codice non abbiano introdotto nuovi difetti o problemi nelle funzionalità già esistenti.

2.2.3.7 Test di Integrazione L'esecuzione dei test d'integrazione è mirata a confermare che le varie componenti del software operino tra loro in modo sinergico e privo di errori.

2.2.3.8 Test di Sistema I test di sistema hanno l'obiettivo di accertare che il comportamento globale del prodotto rispetti integralmente i requisiti software stabiliti durante l'analisi dei requisiti.

2.3 Validazione

La validazione rappresenta il processo finale che conferma in modo definitivo che le caratteristiche del prodotto software siano pienamente conformi ai bisogni dell'utilizzo finale e agli obiettivi d'uso previsti. L'obiettivo principale è rispondere al quesito: *"Did I build the right system?"*

Mentre la verifica analizza la correttezza tecnica dell'esecuzione, la validazione si sposta sul piano dell'efficacia, garantendo che le funzionalità sviluppate siano effettivamente idonee a risolvere le problematiche operative individuate e siano allineate con le aspettative del committente.

2.3.0.1 Processo di Validazione Il gruppo *aLittleByte* ha provveduto a raccogliere, analizzare e formalizzare tutte le necessità espresse dall'azienda proponente *Eggon* all'interno del documento di *Analisi dei Requisiti*. Questa operazione di classificazione è fondamentale per garantire un tracciamento rigoroso delle specifiche e per predisporre i futuri test di accettazione, i quali avranno il compito di accertare che il prodotto software sviluppato rispecchi le richieste del committente.

Gli esiti derivanti da queste procedure di validazione saranno riportati e discussi in modo esaustivo nel *Piano di Qualifica*, all'interno della specifica sezione dedicata ai Test di Accettazione.

Si precisa che, in concomitanza con la pubblicazione dei documenti per l'*RTB*, il progetto si trova in una fase antecedente alla codifica e al collaudo del software. Per tale ragione, lo stato di avanzamento dei test riporterà la dicitura temporanea "Non Implementato". I dati e i risultati effettivi dei test verranno integrati in questa sezione solamente in vista della consegna della *PB*.

2.4 Processi Organizzativi

2.4.1 Descrizione e scopo

In conformità con le direttive tracciate dallo standard ISO/IEC 12207:1997, il processo di gestione comprende l'insieme delle pratiche e delle metodologie necessarie a governare l'intero ciclo di vita del progetto. Lo scopo primario è assicurare il successo dello sviluppo del prodotto software, garantendo che gli obiettivi prefissati vengano raggiunti nel rigoroso rispetto dei vincoli di costo, delle tempistiche stimate e dei requisiti concordati con il proponente.

Per concretizzare tali obiettivi, l'organizzazione del gruppo *aLittleByte* si basa su:

- **Pianificazione:** delle attività da svolgere, della definizione delle scadenze e della suddivisione dettagliata delle mansioni.
- **Monitoraggio e controllo:** Supervisione costante dello stato di avanzamento dei lavori

- **Prevenzione e gestione dei rischi:** Identificazione delle potenziali criticità tecnologiche o organizzative, unita all'adozione di protocolli strutturati per governare le modifiche e per prevenire i possibili imprevisti.
- **Interventi correttivi:** Prontezza e reattività nell'intervenire qualora le metriche di controllo evidenzino deviazioni significative rispetto al Piano di Progetto.
- **Organizzazione del team:** Assegnazione trasparente di ruoli e responsabilità ai membri del gruppo, garantendo al contempo il mantenimento di canali di comunicazione efficaci.

In definitiva, l'applicazione metodica di questi processi garantisce al team di mantenere un progetto solido e tracciabile, costituendo il fondamento necessario per assumere decisioni strategiche in qualsiasi momento dello sviluppo.

2.4.2 Gestione degli sprint

Come già citato in precedenza, il gruppo ha deciso di adottare un approccio iterativo, suddividendo il ciclo di vita del progetto in sprint della durata fissa di due settimane. In concomitanza con l'avvio di ogni nuovo sprint, il team provvede alla rotazione e all'assegnazione dei ruoli che ciascun membro dovrà ricoprire per l'intera durata dello sprint.

L'organizzazione di ogni singola iterazione è scandita da una riunione, organizzata nel seguente modo:

- **Pianificazione:** Il team seleziona gli obiettivi operativi da raggiungere, scompone il lavoro in *task* gestibili e li assegna ai vari componenti, cercando di bilanciare equamente i carichi di lavoro.
- **Sprint Review:** Si analizza lo sprint appena concluso per la verifica concreta dei risultati prodotti. Costituisce l'occasione per valutare il lavoro completato, discutere le criticità tecniche emerse e misurare l'avanzamento effettivo rispetto al piano generale.
- **Sprint Retrospective:** Un momento di auto-valutazione finale, essenziale per il miglioramento continuo del gruppo. Il team si confronta in modo trasparente sulle dinamiche relazionali e metodologiche, individuando i punti di forza da mantenere e le inefficienze organizzative da correggere in vista del ciclo successivo.

2.4.3 Ruoli

2.4.3.1 Responsabile Il membro che assume il ruolo di Responsabile agisce come vero e proprio *Project Manager* del gruppo. Ha la responsabilità di coordinare il lavoro del team, pianificare le iterazioni di sviluppo, ottimizzare l'uso delle risorse e fungere da referente principale per le comunicazioni ufficiali con l'azienda proponente e il corpo docente.

Dal punto di vista strettamente operativo, il Responsabile si fa carico di distribuire e assegnare i compiti (tramite le *Issue*) ai vari componenti del gruppo. Inoltre, è sua competenza diretta la stesura e il costante aggiornamento dei seguenti documenti:

- Verbali (Interni ed Esterni)
- Diario di Bordo
- Piano di Progetto
- Piano di Qualifica

2.4.3.2 Amministratore L'Amministratore rappresenta la figura di supporto metodologico e tecnologico del team. Il suo scopo principale è definire, ottimizzare e mantenere sempre operativi i processi e gli strumenti di lavoro, assicurando che l'intero gruppo possa procedere con le attività in modo fluido e senza impedimenti di natura tecnica o organizzativa.

Oltre a monitorare le scadenze amministrative e a fornire supporto costante ai colleghi, l'Amministratore si fa carico delle seguenti responsabilità operative:

- **Gestione degli ambienti di lavoro**
- **Manutenzione degli strumenti:** Supervisionare e aggiornare periodicamente i software e i canali adottati dal gruppo per la comunicazione e la collaborazione asincrona.
- **Controllo della configurazione:** Garantire la corretta e rigorosa applicazione delle procedure di versionamento per tutti i file e i documenti prodotti dal team.
- **Norme di Progetto:** Curare la stesura, la revisione e il continuo aggiornamento del presente documento (*Norme di Progetto*), assicurandosi che le regole interne e il *Way of Working* siano sempre allineati alle reali necessità del gruppo.

2.4.3.3 Progettista Il Progettista è la mente tecnica dietro alla struttura del prodotto software. Il suo compito principale è ideare l'architettura del sistema, assicurandosi che sia in grado di soddisfare tutti i requisiti (sia funzionali che di qualità) concordati con il proponente. Lavora a stretto contatto con il resto del gruppo per trasformare le richieste astratte in soluzioni pratiche, efficienti e facili da mantenere nel tempo.

Nello specifico, chi ricopre questo ruolo ha la responsabilità di:

- Valutare e decidere quali linguaggi, *framework* e strumenti tecnici sono più idonei per la realizzazione del progetto.
- **Disegnare l'architettura:** Definire come sarà strutturato il sistema a livello generale, individuando i macro-componenti e il modo in cui dovranno comunicare tra loro.
- Supervisionare il lavoro di stesura del codice, assicurandosi che l'implementazione pratica rispetti le specifiche tecniche e non si allontani dall'idea progettuale di partenza.

2.4.3.4 Analista L'Analista rappresenta il punto di incontro tra le esigenze dell'azienda proponente e il lavoro tecnico del team di sviluppo. Il suo compito principale è comprendere a fondo il problema, raccogliere le richieste e trasformarle in regole chiare, complete e ben definite, evitando qualsiasi ambiguità. In pratica, lavora per assicurarsi che tutti sappiano esattamente *cosa* deve essere costruito, prima ancora di decidere *come* farlo.

Nello specifico, chi ricopre questo ruolo si fa carico di:

- **Stesura dell'Analisi dei Requisiti:** Redigere e mantenere costantemente aggiornato il documento che cataloga tutti i casi d'uso e le specifiche del progetto.
- **Gestione dei dubbi e delle modifiche:** Interfacciarsi direttamente con il proponente per risolvere eventuali incertezze, chiedere chiarimenti o concordare variazioni ai requisiti iniziali.
- **Supporto al team:** Collaborare con i progettisti e i programmatori per garantire che le specifiche scritte vengano interpretate nel modo corretto e implementate senza deviare dagli obiettivi concordati.

2.4.3.5 Programmatore Il Programmatore è la figura operativa che materialmente "dà vita" al prodotto software. Il suo compito principale è tradurre l'architettura e le specifiche pensate dal Progettista in codice reale e funzionante. Lavora con l'obiettivo di produrre un software non solo corretto, ma anche pulito, efficiente e facile da mantenere o aggiornare in futuro.

Nello specifico, chi ricopre questo ruolo si fa carico di:

- **Stesura del codice:** Scrivere il codice sorgente rispettando rigorosamente le norme stilistiche e gli standard di qualità definiti dal gruppo.
- **Testing e collaudo preliminare:** Sviluppare ed eseguire i test per verificare che le singole porzioni di codice si comportino come previsto e garantiscano un'elevata affidabilità.
- **Risoluzione dei problemi:** Lavorare a stretto contatto con il resto del team per individuare i difetti, correggere i *bug* tecnici e implementare nuove funzionalità in modo armonico e coerente con il resto del sistema.

2.4.3.6 Verificatore Il Verificatore è il "garante della qualità" all'interno del team. Il suo compito è assicurarsi che ogni elemento prodotto, dal singolo documento di testo fino al codice finale, rispetti fedelmente i requisiti stabiliti e gli standard prefissati.

Nello specifico, chi ricopre questo ruolo si fa carico di:

- **Revisione documentale:** Leggere e controllare i documenti redatti dagli altri membri del team (tramite le *Pull Request*), assicurandosi che siano corretti, completi e privi di ambiguità.
- **Esecuzione dei collaudi:** Pianificare e condurre le varie fasi di *test* sul software (come i collaudi funzionali, di integrazione e di sistema) per verificare il corretto comportamento delle funzionalità.

- **Segnalazione e tracciamento:** Registrare accuratamente i risultati delle verifiche, segnalando al più presto i difetti o le non conformità agli sviluppatori o agli analisti per permetterne la correzione.
- **Miglioramento continuo:** Individuare eventuali punti critici o inefficienze nel modo in cui il team lavora, proponendo soluzioni pratiche per ottimizzare i processi operativi e di sviluppo.

2.4.4 Tracciamento delle Ore

da completare

2.4.5 Tracciamento delle Azioni

Tutte le decisioni operative e le attività da svolgere vengono accuratamente registrate nei verbali, sia a seguito degli incontri interni al gruppo, sia dopo i colloqui esterni con il proponente. Al termine di ogni riunione, il membro designato si occupa di stilare il documento riassuntivo. Le attività da svolgere vengono riportate in fondo al documento nella relativa tabella **To-Do List**.

Per passare dalla teoria alla pratica, ogni compito definito nel verbale viene immediatamente trasformato in un'apposita *Issue* sulla piattaforma GitHub e assegnato al membro o ai membri incaricati. Per garantire una tracciabilità totale, all'interno del verbale stesso viene sempre inserito il collegamento ipertestuale diretto alla *Issue* di riferimento.

Qualora il componente assegnato dovesse riscontrare difficoltà o avere dubbi sull'attività da svolgere, può utilizzare la sezione commenti della singola *Issue* per chiedere delucidazioni al resto del team. Questo approccio ci permette di mantenere le discussioni tecniche ordinate, trasparenti e sempre contestualizzate rispetto al problema specifico da risolvere.

2.4.6 Comunicazione

2.4.6.1 Comunicazioni Interne Le comunicazioni interne avvengono tramite:

- **WhatsApp:** Viene utilizzato per le comunicazioni rapide e coordinamento tra i membri del gruppo.
- **Discussioni GitHub:** Viene utilizzato per discussioni all'interno delle *Issue* e delle *Pull Request*, per garantire il tracciamento delle decisioni. Sono utili in quanto permettono di comunicare solo con i membri del gruppo relativi alla discussione.

2.4.6.2 Comunicazioni Esterne Per interagire con l'azienda proponente e con il corpo docente, il gruppo ha definito dei canali di comunicazione specifici, differenziati in base al livello di formalità e all'urgenza della richiesta:

- **E-mail:** Rappresenta il canale ufficiale per le comunicazioni formali, l'invio della documentazione e gli aggiornamenti periodici sullo stato di avanzamento del progetto. Tutte le comunicazioni via posta elettronica vengono gestite tramite l'indirizzo ufficiale del gruppo (alittlebyte.swe@gmail.com).

- **Telegram:** Impiegato per scambi di messaggi rapidi, chiarimenti immediati e per il coordinamento logistico direttamente con il referente dell'azienda proponente.

2.4.7 Riunioni

2.4.7.1 Riunioni Interne Gli incontri interni del gruppo *aLittleByte* sono pianificati in modo metodico per mantenere un coordinamento efficace e assicurare un avanzamento costante del progetto. Il team ha strutturato i propri momenti di confronto secondo queste modalità:

- **Incontro settimanale:** Di norma, il gruppo si riunisce una volta alla settimana (indicativamente il lunedì). Questo momento di allineamento serve per fare il punto della situazione, organizzare i *task* per i giorni successivi e cercare soluzioni condivise agli ostacoli emersi durante il lavoro individuale.
- **Riunioni straordinarie:** Oltre all'appuntamento fisso, qualora sorgano urgenze, *bug* bloccanti o si debbano prendere decisioni tecniche complesse, i membri interessati possono convocare agilmente degli incontri.
- **Luoghi e piattaforme:** Per comodità organizzativa e per ottimizzare i tempi, le riunioni si svolgono prevalentemente online utilizzando i canali vocali della piattaforma **Discord**.
- **Verbalizzazione:** A garanzia della massima trasparenza operativa, al termine di ogni incontro viene redatto un Verbale Interno.

2.4.7.2 Riunioni Esterne I colloqui con l'azienda proponente sono essenziali per garantire il continuo allineamento sugli obiettivi e raccogliere *feedback* sul lavoro svolto. Tutte le sedute si tengono in via telematica sfruttando la piattaforma **Google Meet**. A conclusione di ogni colloquio, il team si occupa di redigere un Verbale Esterno formale che riepiloga puntualmente le tematiche affrontate, le decisioni prese e le future azioni concordate. Tale documento viene infine inoltrato al referente aziendale per un controllo incrociato e, previa conferma di aderenza con quanto discusso, sottoposto alla sua firma per approvazione definitiva.

3 Qualità del software

Con l'espressione "qualità del software" si indica la totalità delle proprietà di un'entità che concorrono a soddisfare le necessità degli utenti, siano esse dichiarate esplicitamente o sottintese. Tale valutazione si focalizza su due direttrici fondamentali: il valore del prodotto finito e l'efficacia del processo di realizzazione.

L'eccellenza del risultato finale scaturisce invariabilmente dal valore del percorso produttivo intrapreso. Per garantire un software di alto livello, la strategia di valutazione deve essere programmata con cura e non limitata ad una sola verifica finale: tale approccio favorisce una gestione proattiva basata sul monitoraggio costante, superando la logica reattiva volta alla semplice risoluzione delle anomalie.

L'intero processo di valutazione si articola in quattro fasi principali (definizione preventiva delle metriche, misurazione, valutazione dei dati e accettazione finale).

3.1 Qualità del prodotto

Per definire e misurare la qualità intrinseca del software, lo standard di riferimento adottato è la serie di norme ISO/IEC 9126. Questo standard internazionale organizza la qualità in due diverse categorie:

1. **Qualità del Prodotto**, che analizza le seguenti proprietà intrinseche del software:

- Funzionalità;
- Affidabilità,
- Efficienza;
- Usabilità;
- Mantenibilità;
- Portabilità

Al fine di ottenerne una misurazione oggettiva, sono state stabilite specifiche metriche di prodotto, le quali trovano puntuale rendicontazione e misurazione all'interno del *Piano di Qualifica*. Tali metriche consentono di analizzare in termini quantitativi elementi determinanti sia del codice sorgente che dell'applicativo finale, tra cui la funzionalità, l'affidabilità, l'efficienza, l'usabilità, la manutenibilità e la portabilità.

3.2 Qualità del processo

Gli standard di riferimento per la qualità del processo sono l'ISO/IEC 12207. Questo standard fornisce le linee guida per la gestione della qualità nei processi di sviluppo, includendo la pianificazione, il controllo e il miglioramento continuo.

Secondo questo standard la qualità del processo si basa su diversi principi fondamentali:

- **Focalizzazione su Scopo e Risultati (Purpose and Outcomes):** Questo rappresenta il pilastro fondamentale. Per garantire la qualità, ogni fase (dall'approvvigionamento alla creazione, fino alla verifica) deve avere finalità ben definite e generare esiti attesi che siano concreti, misurabili e verificabili.
- **Approccio Modulare (Modularity):** La struttura dei processi deve basarsi su componenti caratterizzate da "forte coesione e debole accoppiamento". Ogni singolo modulo deve concentrarsi su un unico traguardo ed essere autonomo, così da limitare le ripercussioni sull'intero ciclo produttivo in caso di variazioni o criticità in un'area specifica.
- **Attribuzione delle Responsabilità (Responsibility):** Seguendo il principio per cui una responsabilità indistinta equivale a nessuna responsabilità, lo standard richiede che ogni attività sia assegnata a un ruolo preciso (come Sviluppatore, Fornitore, Acquisitore o Verificatore), eliminando incertezze all'interno dell'organizzazione.
- **Personalizzazione dei Processi (Tailoring):** Data la diversità tra i vari progetti software, lo standard prevede il principio del Tailoring. Le aziende devono adattare

i processi alle proprie esigenze, selezionando esclusivamente le procedure pertinenti alla complessità, al rischio e alle risorse disponibili, evitando carichi burocratici superflui.

- **Evoluzione Continua e Tracciabilità:** È essenziale che ogni intervento, anomalia o variazione sia riconducibile al requisito di partenza. Il sistema non è statico, poiché integra cicli di audit, revisione e gestione delle problematiche. Ciò permette al gruppo di lavoro di ottimizzare costantemente le proprie metodologie, avvalendosi spesso di modelli quali CMMI o SPICE per attestare il livello di maturità raggiunto.

In questo progetto, tale obiettivo viene perseguito attraverso una precisa delineazione di metodologie e strumenti operativi. Il controllo costante dell'avanzamento è garantito da sistemi di tracciamento dei problemi (*issue tracking*) e da incontri periodici formalizzati, supportati da un'analisi rigorosa dei requisiti e da verifiche regolari sugli standard qualitativi dei risultati ottenuti.

Al fine di assicurare un monitoraggio imparziale, il gruppo ha stabilito specifiche metriche di processo, dettagliate all'interno del *Piano di Qualifica*. Tali indicatori permettono di misurare l'efficienza del ciclo di sviluppo, focalizzandosi su variabili critiche quali la puntualità nelle consegne, l'aderenza al budget stabilito, la frequenza di anomalie rilevate, l'estensione dei test e la conformità complessiva ai criteri prefissati.

4 Metriche

Questa sezione definisce in modo formale tutte le metriche adottate dal gruppo per quantificare la qualità del processo di sviluppo e del prodotto software finale. I valori di soglia (accettabili e ottimali) e i risultati effettivi delle misurazioni sono demandati e documentati all'interno del *Piano di Qualifica*.

4.1 Metriche per la qualità di processo

Queste metriche (identificate dalla sigla MPC) misurano l'efficacia, l'efficienza e il controllo delle attività necessarie per sviluppare il prodotto.

4.1.1 Processi di Fornitura e Sviluppo

- **MPC-EV - Earned Value:** Rappresenta il valore del lavoro effettivamente completato.
- **MPC-PV - Planned Value:** Rappresenta il valore pianificato del lavoro da completare entro ciascuno sprint.
- **MPC-AC - Actual Cost:** Rappresenta il costo effettivo sostenuto per il lavoro completato in ciascuno sprint.
- **MPC-CPI - Cost Performance Index:** Misura l'efficienza dei costi. Si calcola come:

$$CPI = \frac{EV_{cum}}{AC_{cum}}$$

- **MPC-SPI - Schedule Performance Index:** Misura l'efficienza del tempo. Si calcola come:

$$SPI = \frac{EV_{cum}}{PV_{cum}}$$

- **MPC-EAC - Estimated At Completion:** Stima il costo totale del progetto proiettando la performance corrente. Si calcola come:

$$EAC = \frac{BAC}{CPI}$$

- **MPC-ETC - Estimate To Complete:** Rappresenta la stima dei costi rimanenti al completamento. Si calcola come:

$$ETC = EAC - AC_{cum}$$

- **MPC-TEAC - Time Estimate At Completion:** Stima la durata totale del progetto a partire dalla performance di avanzamento. Si calcola come:

$$TEAC = \frac{\text{Durata pianificata}}{SPI}$$

- **MPC-RSI - Requirements Stability Index:** Calcolato confrontando il numero di requisiti aggiunti, modificati o eliminati rispetto alla versione iniziale. Formula:

$$RSI = \left(1 - \frac{N_{modificati} + N_{eliminati} + N_{aggiunti}}{N_{totali_iniziali}} \right) \times 100\%$$

4.1.2 Processi di Supporto

- **MPC-IG - Indice di Gulpease:** Misura la leggibilità dei documenti in lingua italiana. Formula:

$$G = 89 - \frac{10 \cdot L}{P} + \frac{300 \cdot F}{P}$$

- **MPC-CO - Correttezza ortografica:** Analisi automatica della correttezza ortografica per individuare errori di battitura o sintassi.
- **MPC-CC - Code Coverage:** Misura la percentuale di codice sorgente eseguita durante l'esecuzione dei test automatici.
- **MPC-TSR - Test Success Rate:** Percentuale di test automatici che hanno restituito un esito positivo in fase di esecuzione.
- **MPC-QMS - Quality Metrics Satisfied:** Misura la percentuale di metriche di processo che soddisfano la soglia accettabile. Formula:

$$QMS = \frac{\text{Metriche soddisfatte}}{\text{Metriche misurate}} \times 100\%$$

4.1.3 Processi Organizzativi

- **MPC-TE - Time Efficiency:** Calcolato confrontando le ore produttive effettivamente rendicontate con il totale delle ore disponibili. Formula:

$$TE = \frac{\text{Ore produttive}}{\text{Ore totali disponibili}} \times 100\%$$

- **MPC-PRI - Percentuale Rischi Inattesi:** Confronta i rischi effettivamente manifestatisi con quelli identificati nel registro dei rischi. Formula:

$$PRI = \frac{N_{rischi_inattesi}}{N_{rischi_totali_manifestati}} \times 100\%$$

- **MPC-LE - Labor Efficiency:** Confronta le ore dedicate ad attività a valore aggiunto con il totale delle ore lavorate. Formula:

$$LE = \frac{\text{Ore a valore aggiunto}}{\text{Ore totali lavorate}} \times 100\%$$

4.2 Metriche per la qualità di prodotto

Queste metriche (identificate dalla sigla MPD) misurano le proprietà interne ed esterne del software finale per garantire che il sistema rispetti i requisiti.

- **Funzionalità:**
 - **MPD01:** Requisiti obbligatori soddisfatti
 - **MPD02:** Requisiti opzionali soddisfatti.
 - **MPD03:** AI Acceptance Rate (rating $\geq 3/5$)
- **Affidabilità:**
 - **MPD04:** Branch Coverage
 - **MPD05:** Defect Density (Densità dei difetti per KLOC).
- **Efficienza:**
 - **MPD06:** Latenza interfaccia utente.
 - **MPD07:** Latenza generazione testo AI Assistant.
 - **MPD08:** Latenza generazione immagine AI Assistant.
 - **MPD09:** Latenza analisi documentale AI Co-Pilot.
- **Usabilità:**
 - **MPD10:** Profondità Massima di Navigazione (numero di livelli).
 - **MPD11:** Learning Time (tempo di apprendimento per le funzioni principali).

- **Mantenibilità:**

- **MPD12:** Code Smell.
- **MPD13:** Coefficient of Coupling.
- **MPD14:** Cyclomatic Complexity (complessità per singolo metodo).

- **Portabilità:**

- **MPD15:** Compatibilità Cross-Browser.